

DirZero C++ Rewrite - Team Task Assignment

1. Project Objective

Rebuild the DirZero desktop app in C++ with Qt6 and a proper remote SSH/SFTP backend, while keeping the working Python app as the reference and the C++ rewrite as the production target.

The rewrite must include:

- Tailscale peer discovery
 - remote reachability checks
 - SSH authentication
 - SFTP remote file browsing
 - create / rename / delete / move / copy operations
 - drag-and-drop / clipboard behavior
 - themes and UI polish
 - permission and access-control handling
 - clean architecture and testable modules
-

2. Team Structure

- Shubhneek(integration)
 - Dhwani
 - Ritesh
 - Eka
 - Jaideep
-

3. Core Working Rule

Do not let one person become the single bottleneck.

Each module must have:

- one owner(as in responsible for development and handling)
- one reviewer/support member
- a small deliverable
- a demo or test result
- a pull request before merge

The lead remains responsible for final architecture, triage, integration, and approval of all major merges.

4. Ownership Mapping

Shubhneek - Team Lead + Core Backend Architecture

Responsible for:

- overall architecture and module boundaries
- CMake + build system setup
- final app assembly and integration
- core permission/access control design
- proving the backend architecture works under load
- final review and merge gatekeeping

Primary deliverables:

1. project skeleton and build system
2. shared core data model and interfaces
3. permission policy framework for the rewrite
4. final integration of all modules into a working app
5. high-risk bug fixing and architecture decisions

Suggested features:

- secure access-check design before file operations
- permission state modeling for SSH/SFTP actions
- architecture that keeps backend logic independent from UI
- final cross-module consistency checks

Recommended module names:

- `PermissionPolicy.hpp/.cpp`
- `AuthState.hpp`
- `AccessRule.hpp`
- `Dir2ZeroWindow.hpp/.cpp` (final integration layer)

Related work:

- create the architecture and final integration point
- reviews network state model and filesystem contracts
- handles the final assembly of the app with the UI shell

Dhwani - Permission, Security, and Credential Control

Her responsibilities:

- credential storage and validation
- permission policy design
- access checks for dangerous operations
- secure handling of auth and file operation rules

Her main scope:

- `CredentialStore`
- `PermissionPolicy`
- `AuthState`
- access checks for create, rename, delete, move, copy

- secure file operation gating logic

This is a dedicated backend/security task and is intentionally not treated as a side task.

Ritesh - Discovery, Connectivity, and Machine Health

His responsibilities:

- Tailscale peer discovery
- remote reachability checks
- machine metadata and status parsing
- health state monitoring for online/offline and SSH availability

His main scope:

- `MachineInfo`
- `MachineState`
- `TailscaleDiscovery`
- `ProbeWorker`
- connectivity and status refresh logic

This is the network/diagnostics layer and is the backbone for machine availability in the app.

Eka - UI Layer and User Interaction

Her responsibilities:

- machine panels and main window UI
- tree view interaction
- action flow for connect, auth, file operations, and navigation
- theming and UI polish

Her main scope:

- `MachinePanelWidget`
 - `RemoteFileTreeView`
 - `Dir2ZeroWindow`
 - `MachineAuthBar`
 - `ThemeManager`
-

Jaideep - Shared Backend Support and Integration Helpers

His responsibilities:

- help keep shared data contracts consistent
- support backend utilities and metadata formatting
- help with model glue and app integration stability
- support the UI/backend boundary without owning the full visual layer

His main scope:

- RemoteClipboard
 - ResponsiveGridContainer
 - auxiliary metadata helpers
 - state and formatting utility support
 - integration debugging support
-

5. Specific Feature Tasks

Phase 1 - Foundation

Task 1: Project skeleton and build system

Owner: Team Lead

- Create the C++ project structure
- Setup CMake and Qt6 dependencies
- Make the app compile on Linux
- Keep the first compile clean and minimal

Task 2: Shared base types

Owner: Ritesh

- MachineInfo
- MachineState
- basic enums and metadata structures
- status model definitions

Task 3: Minimal window build

Owner: Eka

- Create an empty but working Qt main window
 - confirm UI loads correctly
 - verify theme application works
-

Phase 2 - Core Discovery and Network

Task 4 - Tailscale peer discovery

Owner: Ritesh

- parse Tailscale output
- list connected peers
- create machine metadata objects

Task 5 - Remote reachability checks

Owner: Ritesh

- check SSH port 22
- classify online/unreachable/offline
- maintain health state

Task 6 - App state wiring

Owner: Shubhneek

- combine machine discovery and health into a clean app state model
 - ensure UI and backend agree on status values
-

Phase 3 - Security and Permissions

Task 7 - Secure credential storage

Owner: Dhwani

- save username, key path, and password securely
- create a local credential store
- enforce safe permissions on stored config files

Task 8 - Permission policy module

Owner: Dhwani

- define access rules
- enforce blocked actions on unauthorized operations
- provide shared permission decisions to the backend

Task 9 - Operation gate checks

Owner: Dhwani

- validate create, rename, delete, move, and transfer calls
 - return failures cleanly before dangerous operations happen
-

Phase 4 - Filesystem and Remote Operations

Task 10 - FS tree model

Owner: Ritesh

- build the remote directory tree representation
- support lazy loads and nested folders
- keep node and path state consistent

Task 11 - SFTP connection and listing

Owner: Ritesh

- connect to SSH/SFTP sessions
- list remote directories
- handle connection failure states properly

Task 12 - File operations and transfers

Owner: Dhvani

- implement create/delete/rename/move/copy logic
 - support transfer progress and error behavior
 - gate actions by permission policy
-

Phase 5 - UI and user experience

Task 13 - Machine panel UI

Owner: Eka

- create the panel card and machine state display
- show connect/auth status and actions

Task 14 - Main window layout

Owner: Eka

- create the multi-panel shell
- integrate state updates and user actions

Task 15 - Theme system

Owner: Eka

- load and apply stylesheet themes
- support runtime switching

Task 16 - Clipboard and drag/drop flow

Owner: Eka + Jaideep

- handle clipboard actions and drag/drop interactions
 - keep operation flow consistent with backend state
-

Phase 6 - Final Integration and QA

Task 17: Integration and bug fixing

Owner: Team Lead

- merge all feature branches
- resolve API mismatches
- clean compile and runtime state

Task 18: QA + regression pass

Owner: All members

- test broken flows
- test permission denial cases
- test offline machine states
- test remote file operations
- test theme switching and resizing

Task 19: Release checklist

Owner: Team Lead

- compile check
- no runtime crash on connect/disconnect
- no thread deadlock during directory listing
- correct permission failure messages
- code review done on all modules

6. Final Review Model

Final ownership stays with the lead, but each member handles their feature area independently:

- Shubhneek: final architecture and integration quality
- Dhvani: security and permission correctness
- Ritesh: machine health and discovery stability
- Eka: UI behavior and interaction flow
- Jaideep: support layer and contract consistency

7. Submission Expectation

Each member should provide:

- owned module(s)
- a working implementation or draft branch
- a small proof of functionality
- a short status summary
- known limitations and missing pieces

This keeps the rewrite realistic and reviewable without everyone depending on the lead for every task.

This structure keeps the work distributed, reviewable, and realistic for a 5-person rewrite.